

# 蓝桥杯30天算法冲刺集训



# Day-27 数论I

## 1. 整除

整除的定义：设  $a, b \in \mathbf{Z}$ ,  $a \neq 0$ 。如果  $\exists q \in \mathbf{Z}$ , 使得  $b = aq$ , 那么就说  $b$  可被  $a$  整除, 记作  $a \mid b$ , 读作  $a$  整除  $b$ , 或者  $b$  可被  $a$  整除;  $b$  不被  $a$  整除记作  $a \nmid b$ 。

整除的性质:

- $a \mid b \iff -a \mid b \iff a \mid -b \iff |a| \mid |b|$
- $a \mid b \wedge b \mid c \implies a \mid c$
- $a \mid b \wedge a \mid c \iff \forall x, y \in \mathbf{Z}, a \mid (xb + yc)$
- $a \mid b \wedge b \mid a \implies b = \pm a$
- 设  $m \neq 0$ , 那么  $a \mid b \iff ma \mid mb$ 。
- 设  $b \neq 0$ , 那么  $a \mid b \implies |a| \leq |b|$ 。
- 设  $a \neq 0, b = qa + c$ , 那么  $a \mid b \iff a \mid c$ 。

约数 (因数): 若  $a \mid b$ , 则称  $b$  是  $a$  的倍数,  $a$  是  $b$  的约数。

0 是所有非 0 整数的倍数。对于整数  $b \neq 0$ ,  $b$  的约数只有有限个。

平凡约数 (平凡因数): 对于整数  $b \neq 0$ ,  $\pm 1, \pm b$  是  $b$  的平凡约数。当  $b = \pm 1$  时,  $b$  只有两个平凡约数。

对于整数  $b \neq 0$ ,  $b$  的其他约数称为真约数 (真因数、非平凡约数、非平凡因数)。

约数的性质:

- 设整数  $b \neq 0$ 。当  $d$  遍历  $b$  的全体约数的时候,  $\frac{b}{d}$  也遍历  $b$  的全体约数。
- 设整数  $b > 0$ , 则当  $d$  遍历  $b$  的全体正约数的时候,  $\frac{b}{d}$  也遍历  $b$  的全体正约数。

在具体问题中, **如果没有特别说明, 约数总是指正约数。**

## 带余数除法

余数的定义: 设  $a, b$  为两个给定的整数,  $a \neq 0$ 。设  $d$  是一个给定的整数。那么, 一定存在唯一的一对整数  $q$  和  $r$ , 满足  $b = qa + r, d \leq r < |a| + d$ 。

无论整数  $d$  取何值,  $r$  统称为余数。  $a \mid b$  等价于  $a \mid r$ 。

一般情况下,  $d$  取 0, 此时等式  $b = qa + r, 0 \leq r < |a|$  称为带余数除法 (带余除法)。这里的余数  $r$  称为最小非负余数。

余数往往还有两种常见取法：

绝对最小余数： $d$  取  $a$  的绝对值的一半的相反数。即  $b = qa + r, -\frac{|a|}{2} \leq r < |a| - \frac{|a|}{2}$ 。

最小正余数： $d$  取 1。即  $b = qa + r, 1 \leq r < |a| + 1$ 。

带余数除法的余数只有最小非负余数。**如果没有特别说明，余数总是指最小非负余数。**

余数的性质：

- 任一整数被正整数  $a$  除后，余数一定是且仅是 0 到  $(a - 1)$  这  $a$  个数中的一个。
- 相邻的  $a$  个整数被正整数  $a$  除后，恰好取到上述  $a$  个余数。特别地，一定有且仅有一个数被  $a$  整除。

---

**计算机中的取余：**所有编程语言和计算器都遵循了尽量让 **商尽量靠近0** 的原则，即  $5\%(-3)$  的结果为 2 而不是  $-1$ ， $(-5)\%3$  的结果是  $-2$  而不是 1。

比如： $-5\%3 = -2$ ， $-5\% - 3 = -2$ ， $5\% - 3 = 2$

即：计算  $a\%b$ ，先看成两个都是正数，然后取余，如果  $a$  是正数，结果就是正数，如果  $a$  是负数，结果就是负数， $b$  的正负不影响。

如果要让计算机计算出数学上的余数，可以如此实现： $(a\%b + abs(b))\%b$

## 2. 同余

**同余** 是伟大的德国数学家高斯在 19 世纪初期发明的语言，这种语言使我们在研究整除性关系的时候变得和研究方程那样容易。

同余的定义：设整数  $m \neq 0$ 。若  $m \mid (a - b)$ ，称  $m$  为模数（模）， $a$  同余于  $b$  模  $m$ ， $b$  是  $a$  对模  $m$  的剩余。记作  $a \equiv b \pmod{m}$ 。

否则， $a$  不同余于  $b$  模  $m$ ， $b$  不是  $a$  对模  $m$  的剩余。记作  $a \not\equiv b \pmod{m}$ 。

这样的等式，称为模  $m$  的同余式，简称同余式。

根据整除的性质，上述同余式也等价于  $a \equiv b \pmod{-m}$ 。

**如果没有特别说明，模数总是正整数。**

式中的  $b$  是  $a$  对模  $m$  的剩余，这个概念与余数完全一致。通过限定  $b$  的范围，相应的有  $a$  对模  $m$  的最小非负剩余、绝对最小剩余、最小正剩余。

同余的基本性质：

- 自反性： $a \equiv a \pmod{m}$ 。
- 对称性：若  $a \equiv b \pmod{m}$ ，则  $b \equiv a \pmod{m}$ 。
- 传递性：若  $a \equiv b \pmod{m}$ ， $b \equiv c \pmod{m}$ ，则  $a \equiv c \pmod{m}$ 。
- 同加性：若  $a \equiv b \pmod{m}$ ，则  $a + c \equiv b + c \pmod{m}$ 。

- 同乘性：若  $a \equiv b \pmod{m}$ ，则  $a \times c \equiv b \times c \pmod{m}$ 。
- 同幂性：若  $a \equiv b \pmod{m}$ ，则  $a^c \equiv b^c \pmod{m}$ 。
- 线性运算：若  $a, b, c, d \in \mathbf{Z}, m \in \mathbf{N}^*, a \equiv b \pmod{m}, c \equiv d \pmod{m}$  则有：

$$* \quad a \pmod{m} c \equiv b \pmod{m} d \pmod{m}。$$

- $a \times c \equiv b \times d \pmod{m}$ 。
- 若  $a, b \in \mathbf{Z}, k, m \in \mathbf{N}^*, a \equiv b \pmod{m}$ ，则  $ak \equiv bk \pmod{mk}$ 。
- 若  $a, b \in \mathbf{Z}, d, m \in \mathbf{N}^*, d \mid a, d \mid b, d \mid m$ ，则当  $a \equiv b \pmod{m}$  成立时，有  $\frac{a}{d} \equiv \frac{b}{d} \pmod{\frac{m}{d}}$ 。
- 若  $a, b \in \mathbf{Z}, d, m \in \mathbf{N}^*, d \mid m$ ，则当  $a \equiv b \pmod{m}$  成立时，有  $a \equiv b \pmod{d}$ 。
- 若  $a, b \in \mathbf{Z}, d, m \in \mathbf{N}^*$ ，则当  $a \equiv b \pmod{m}$  成立时，有  $\gcd(a, m) = \gcd(b, m)$ 。  
◦ 若  $d$  能整除  $m$  及  $a, b$  中的一个，则  $d$  必定能整除  $a, b$  中的另一个。

同余运算的四则运算：

- $(a + b) \pmod{c} = ((a \pmod{c}) + (b \pmod{c})) \pmod{c}$
- $(a - b) \pmod{c} = ((a \pmod{c}) - (b \pmod{c})) \pmod{c}$
- $(a \times b) \pmod{c} = ((a \pmod{c}) \times (b \pmod{c})) \pmod{c}$
- $(a \div b) \pmod{c} = ((a \pmod{c}) \times (b^{-1} \pmod{c})) \pmod{c}$ ，其中  $b^{-1}$  称作  $b$  在模  $c$  意义下的 **乘法逆元**
- 如果我们能够找到一个数满足  $bb^{-1} \equiv 1 \pmod{c}$ ，就能定同余的除法运算了。所以将除法定义为，如果  $b$  和  $c$  互质，则可以在  $c$  的缩剩余系下找到唯一的一个数  $d = b^{-1}$  满足  $bd = 1 \pmod{c}$ ，这个数字  $d$  被称为在模  $c$  意义下  $b$  的 **乘法逆元**。如果  $b$  和  $c$  不互质，那就相当于熟悉的普通的四则运算中的除以 0，可以认为这是未定义的。

## 3. 最大公约数与最小公倍数

### 3.1 最大公约数

最大公约数即为 Greatest Common Divisor，常缩写为 gcd。

一组整数的公约数，是指同时是这组数中每一个数的约数的数。 $\pm 1$  是任意一组整数的公约数。

一组整数的最大公约数，是指所有公约数里面最大的一个。

#### 欧几里德算法：辗转相除法求最大公约数

辗转相除法是一种算法，也称 欧几里德算法(Euclid)。

如果我们已知两个数  $a$  和  $b$ ，如何求出二者的最大公约数呢？欧几里德算法显示了，可以用如下等式去递归求解——

$$\gcd(a, b) = \gcd(b, a \% b)$$

上述等式的 **证明** 如下:

不妨设  $a > b$ , 我们发现如果  $b$  是  $a$  的约数, 那么  $b$  就是二者的最大公约数。下面讨论不能整除的情况, 即  $a = b \times q + r$ , 其中  $r < b$ 。

我们通过证明可以得到  $\gcd(a, b) = \gcd(b, a \bmod b)$ , **首先证明  $a, b$  的公约数一定也是  $b, a \bmod b$  的公约数。**

设  $a = bk + r$ , 显然有  $r = a \bmod b$ 。设有  $d$  是  $a, b$  的公约数, 即  $d \mid a, d \mid b$ , 则  $r = a - bk, \frac{r}{d} = \frac{a}{d} - \frac{b}{d}k$ 。

由右边的式子可知  $\frac{r}{d}$  为整数, 即  $d \mid r$ , 所以对于  $a, b$  的公约数  $d$ , 它也会是  $b, r = a \bmod b$  的公约数。

然后再证明  $b, a \bmod b$  的公约数也是  $a, b$  的公约数:

设  $d \mid b, d \mid (a \bmod b)$ , 我们还是可以像之前一样得到以下式子  $\frac{a \bmod b}{d} = \frac{a}{d} - \frac{b}{d}k, \frac{a \bmod b}{d} + \frac{b}{d}k = \frac{a}{d}$ 。

因为左边式子显然为整数, 所以  $\frac{a}{d}$  也为整数, 即  $d \mid a$ , 所以  $b, a \bmod b$  的公约数也是  $a, b$  的公约数。

综合上面结论可知:  $a, b$  和  $b, a \bmod b$  的公约数都是相同的, 所以其最大公约数也一定相同。

所以得到式子  $\gcd(a, b) = \gcd(b, a \bmod b)$

既然得到了  $\gcd(a, b) = \gcd(b, r)$ , 这里两个数的大小是不会增大的, 那么我们也得到了关于两个数的最大公约数的一个递归求法。其代码可以非常简洁的写成如下形式:

```
int gcd(int a, int b) {  
    return b == 0 ? a : gcd(b, a % b);  
}
```

## 3.2 最小公倍数

最小公倍数 (Least Common Multiple, LCM) 。

一组整数的公倍数, 是指同时是这组数中每一个数的倍数的数。0 是任意一组整数的公倍数。

一组整数的最小公倍数, 是指所有正的公倍数里面, 最小的一个数。

### 两个数的最小公倍数

设  $a = p_1^{k_{a1}} p_2^{k_{a2}} \cdots p_s^{k_{as}}, b = p_1^{k_{b1}} p_2^{k_{b2}} \cdots p_s^{k_{bs}}$

我们发现, 对于  $a$  和  $b$  的情况, 二者的最大公约数等于

$$p_1^{\min(k_{a1}, k_{b1})} p_2^{\min(k_{a2}, k_{b2})} \cdots p_s^{\min(k_{as}, k_{bs})}$$

最小公倍数等于

$$p_1^{\max(k_{a_1}, k_{b_1})} p_2^{\max(k_{a_2}, k_{b_2})} \cdots p_s^{\max(k_{a_s}, k_{b_s})}$$

由于  $k_a + k_b = \max(k_a, k_b) + \min(k_a, k_b)$

所以得到结论是：  $\gcd(a, b) \times \text{lcm}(a, b) = a \times b$

要求两个数的最小公倍数，先求出最大公约数即可，即：  $\text{lcm}(a, b) = a \times b / \gcd(a, b)$ 。

需要注意的是：在代码实现的过程中，上面的式子推荐写成先除后乘的形式。（为什么？）

## 多个数的最小公倍数

可以发现，当我们求出两个数的  $\gcd$  时，求最小公倍数是  $O(1)$  的复杂度。那么对于多个数，我们其实没有必要求一个共同的最大公约数再去处理，最直接的方法就是，当我们算出两个数的  $\gcd$ ，或许在求多个数的  $\gcd$  时候，我们将它放入序列对后面的数继续求解，那么，我们转换一下，直接将最小公倍数放入序列即可。

## 3.3 互素

两个整数互素（既约）的定义：若  $\gcd(a_1, a_2) = 1$ ，则称  $a_1$  和  $a_2$  互素（既约）。

多个整数互素（既约）的定义：若  $\gcd(a_1, \dots, a_k) = 1$ ，则称  $a_1, \dots, a_k$  互素（既约）。

多个整数互素，不一定两两互素。例如 6、10 和 15 互素，但是任意两个都不互素。

互素的性质与最大公约数理论：裴蜀定理（Bézout's identity）。见后面的裴蜀定理。

两个数的最大公约数的一个递归求法。

## 4. 素数

### 算术基本定理（唯一分解定理）

唯一分解定理的通俗语言描述：每个大于 1 的正整数都可以被唯一地写成若干个素数的乘积，乘积中的素因子按照非降序排列。

该定理告诉了我们一个重要的结果：**素数是整数的乘法运算的构成单元。**

算术基本引理：

设  $p$  是素数， $p \mid a_1 a_2$ ，那么  $p \mid a_1$  和  $p \mid a_2$  至少有一个成立。

算术基本引理是素数的本质属性，也是素数的真正定义。

算术基本定理（唯一分解定理）：

设正整数  $a$ ，那么必有表示：

$$a = p_1 p_2 \cdots p_s$$

其中  $p_j (1 \leq j \leq s)$  是素数。并且在不计次序的意义下，**该表示唯一**。

标准素因数分解式：

将上述表示中，相同的素数合并，可得：

$$a = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_s^{\alpha_s}, p_1 < p_2 < \cdots < p_s$$

称为正整数  $a$  的标准素因数分解式。

算术基本定理和算术基本引理，两个定理是等价的。

## 由唯一分解定理推出的性质

设正整数  $n = p_1^{k_1} p_2^{k_2} \cdots p_m^{k_m}$ ，其中  $p_1, p_2, \dots, p_m$  是互不相同的质数， $k_1, k_2, \dots, k_m$  是正整数。那么有如下推论：

- 约数个数  $d(n) = (1 + k_1)(1 + k_2) \cdots (1 + k_m) = \prod_{i=1}^m (1 + k_i)$

证明：可以用分步乘法原理证明该公式，略。

- 约数  $S(n) = (1 + p_1 + p_1^2 + \cdots + p_1^{k_1})(1 + p_2 + p_2^2 + \cdots + p_2^{k_2}) \cdots (1 + p_m + p_m^2 + \cdots + p_m^{k_m}) = \prod_{i=1}^m \sum_{j=0}^{k_i} p_i^j$  和

证明：在计算约数时，对素因子  $p_i$  可以取  $0 \sim k_i$  个。对上式进行分解后就可以发现，该式分解后每一项都是  $n$  的一个约数，且因为是分步乘法原理的组合，因此不重不漏，所以  $S(n)$  是  $n$  的约数和。

- 小于或等于  $n$  的正整数中与  $n$  互质的数的数目，也即欧拉函数  $\varphi(n) = n(1 - \frac{1}{p_1})(1 - \frac{1}{p_2}) \cdots (1 - \frac{1}{p_m})$

证明：不好证，建议直接记忆。

## 蓝桥杯真题

### 取模（蓝桥杯Python2022B组国赛）

数论思维巧题。

首先我们直接看，发现要枚举  $m$  个数，似乎复杂度无法降低。

但是如果要是使得

$$res_i = n \bmod p \quad (p \in [1, m])$$

得出来的结果两两不相同，那么肯定是下列式子对于任意  $p \in [1, m]$  都成立

$$p - 1 = n \bmod p \quad (p \in [1, m])$$

我们只需要模拟一下就可以发现，1一定会出现并且模数肯定是0，如果2要出现，那么模数必须是1，否则就跟1的模数一样了。接着如果模2的模数不是0而是1，那么3的模数也必须得是2，这样依次类推即可。

那么这么直接暴力找可以不可以呢？答案是可以的，下面证明迭代次数不会过大，当然蓝桥杯里面你直接写个暴力然后自己测几个大数据能过就可以了，因为毕竟OI赛制，不是跟ACM一样可能还会直接TLE然后0分，但是这题完全做完还是需要去证明一下迭代次数不会过大的。

首先我们定义

$$L = LCM(1, 2, 3, \dots, m)$$

然后我们会发现

$$L - 1 \equiv p_i - 1 \pmod{p_i}, \quad 1 \leq p_i \leq m$$

接着可以发现

$$LCM(1, 2, \dots, 30) > 10^9$$

所以迭代次数不会很多，因为两两不同模数的情况是 $L - 1$ ，但是 $L$ 的增长速度非常快，对于题目给定的 $1 \leq n \leq 10^9$ ，最多的 $m$ 迭代暴力尝试次数不会超过30

```
#include <bits/stdc++.h>
using namespace std;
void solve()
{
    int n,m;
    cin>>n>>m;
    for(int i=2;i<=m;i++)
        if(n%i!=i-1)
        {
            puts("Yes");
            return;
        }
    puts("No");
}
int main()
{
    int T;
    cin>>T;
    while(T--) solve();
}
```



## 序列求和 (蓝桥杯C/C++2019A组国赛)

上面的讲义也说明了

设正整数  $n = p_1^{k_1} p_2^{k_2} \cdots p_m^{k_m}$ ，其中  $p_1, p_2, \dots, p_m$  是互不相同的质数， $k_1, k_2, \dots, k_m$  是正整数。那么对于约数个数  $d(n)$  有如下推论：

$$d(n) = (1 + k_1)(1 + k_2) \dots (1 + k_m) = \prod_{i=1}^m (1 + k_i)$$

那么我们是不是直接用  $d(n)$  这个递推式子，找到约数个数最小的质因数的幂次集合，然后拼凑质因数即可。

那么我们可以反向的再分解质因数，也就是假设我们要求约数个数是  $n = 12$  的最小正整数可以列举一下

$$12 = 6 \times 2$$

$$12 = 3 \times 4$$

$$12 = 3 \times 2 \times 2$$

那么我们发现用  $12 = 3 \times 2 \times 2$  来看，实际上如果约数个数是12个的话，那么质因数的幂次一定是  $(3 - 1) = 2, (2 - 1) = 1, (2 - 1) = 1$  个的。

那么我们只需要贪心的从小到大枚举这个质因数就可以了，也就是可以找到约数个数是12的最小正整数是  $2^2 \times 3^1 \times 5^1 = 60$

这样我们只需要用DFS去求解一下各个可能的乘积情况就可以了。

```
#include <bits/stdc++.h>
using namespace std;
typedef unsigned long long ll;
const ll N=1e5,inf=1e18;
bool vis[N+10];
ll prime[N+10],tot=0,mi;
void get_prime() // 素数筛
{
    memset(vis,1,sizeof(vis));
    vis[0]=vis[1]=0;
    for(int i=2;i<=N;i++)
    {
        if(vis[i])prime[tot++]=i;
        for(int j=0;j<tot&&i*prime[j]<=N;j++)
        {
            vis[i*prime[j]]=0;
            if(i%prime[j]==0)break;
        }
    }
}
```

```

ll qpow(ll a,ll b) // 快速幂
{
    ll s=1;
    while(b)
    {
        if(b&1)s=s*a;
        b/=2;
        a*=a;
    }
    return s;
}

ll dfs(ll x,vector<ll>g) // 将x分解成若干个数的乘积
{
    if(vis[x]||x==1)return qpow(2,x-1); // 特判质数和1
    for(ll i=2;i*i<=x;i++)
    {
        if(x%i==0)
        {
            ll t=x/i; // 大的那个数
            g.push_back(i);

            // 更新mi
            vector<ll>tmp=g;
            tmp.push_back(t);
            sort(tmp.begin(),tmp.end(),greater<ll>());
            ll s=1;
            for(int j=0;j<tmp.size();j++)
                s*=qpow(prime[j],tmp[j]-1);
            mi=min(mi,s);

            dfs(t,g);
            g.pop_back();
        }
    }
    return mi;
}

int main()
{
    ios::sync_with_stdio(false);
    get_prime();
    ll ans=0;
    vector<ll>g;
    for(int i=1;i<=60;i++)
    {
        mi=inf;
        g.clear();
    }
}

```

```
    ll ts=dfs(i,g);
    ans+=ts;
    //printf("i=%d ts=%lld ans=%lld\n",i,ts,ans);
}
printf("%lld\n",ans);
return 0;
}
```

//292809912969717649